

Resumen del capítulo: Las bases de lanzamiento de scripts

Acceder a la línea de comando desde tu máquina local

Puedes ejecutar comandos **localmente** sin conectarte a servicios cloud (en la nube).

Los usuarios que trabajan en Linux tienen acceso a la línea de comando.

macOS se basa en Linux así que posee todas sus características especiales. Si trabajas en macOS, ve a la carpeta `Applications/Utilities` y busca `Terminal`. Esa es la línea de comando en este sistema operativo.

Si trabajas en Windows, instala el sistema operativo "paralelo" completamente funcional Ubuntu, que es de la familia Linux. Para ello:

1. Presiona "Start" ("Inicio") y abre Microsoft Store.
2. Busca Ubuntu.
3. Verás la ventana de la app Ubuntu; presiona Instalar. Ubuntu se instalará en tu PC.
4. Reinicia el sistema. Después presiona "Start" e introduce `Ubuntu`.
5. Lanza Ubuntu: espera hasta que el sistema finalice la configuración inicial. Después verás la ventana de terminal.

Una cosa a tener en cuenta cuando trabajes con Ubuntu en Windows es que Ubuntu tiene acceso al sistema de archivos pero organiza las rutas de archivo de una forma diferente a Windows. Por ejemplo, puedes encontrar la carpeta raíz drive C en Windows por la ruta `C:\`. Pero en Ubuntu los archivos de esta carpeta tienen la dirección `/mnt/c/`.

Comandos básicos de interfaz en Linux

Los comandos más comunes en Linux son:

- `whoami` devuelve el nombre de usuario
- `cd` permite cambiar la carpeta actual
- `mkdir` crea una carpeta
- `rm` elimina un archivo o carpeta vacía
- `rm -r` elimina una carpeta y los archivos que contiene
- `cat` imprime los contenidos del archivo
- `echo` imprime un texto o los contenidos de una variable

Esta lista de comandos es más que suficiente para configurar una automatización básica.

Lanzar un script desde la línea de comandos

Para automatizar el proceso necesitas enseñar a tu PC a lanzar programas con parámetros de entrada por horario. Ahí es donde entran en juego los scripts de Python.

Descarga un editor de texto (por ejemplo, [Sublime Text](#) (*materiales en inglés*) o [Notepad++](#) (*materiales en inglés*)) para editar tus scripts de Python. Crea una fila vacía en el editor de texto y guárdala como `test.py`.

Inserta un script mínimo de Python en el archivo (se ejecutará en cualquier sistema operativo):

```
#!/usr/bin/python

if __name__ == "__main__":
    print('Hello world.')
```

El sistema producirá el mensaje: `'Hello world.'` ('Hola mundo.')

Tu script de Python consiste en varios componentes:

- `#!/usr/bin/python` le dice al SO en qué lenguaje está escrito el script
- `# -*- coding: utf-8 -*-` comunica que el script utiliza `UTF-8` de codificación de caracteres. Estrictamente hablando, no necesitas indicar esto en Python 3, es predeterminado.
- `if __name__ == "__main__"` es una condición que contiene la parte principal del script que se ejecuta.

En Python los scripts se pueden llamar de dos formas:

- Como el programa principal si el script es lanzado directamente desde la línea de comandos
- Como un modulo importado: a través del comando `import` dentro de otro archivo

Ejecutar un script desde la línea de comandos es fácil:

```
python /script_path/script_name.py
```

Importa las librerías `getopt` y `sys`. `getopt` (del inglés "get options", "conseguir opciones") lee los parámetros input u **opciones**. `sys` importa las funciones del sistema. Necesitarás su función `sys.exit()` en los scripts: para la ejecución del script si se lanza sin parámetros de entrada (input). Aquí está el código del script:

```
#!/usr/bin/python

# Importa las librerías necesarias
import sys
import getopt
```

```

if __name__ == "__main__":

    # Establece el formato de los parámetros input
    unixOptions = "s:e:"
    gnuOptions = ["start_dt=", "end_dt="]

    # Obtén la string con parámetros input
    fullCmdArguments = sys.argv
    argumentList = fullCmdArguments[1:]

    # Comprueba si los parámetros input coinciden con el formato
    # indicado en unixOptions y gnuOptions
    try:
        arguments, values = getopt.getopt(argumentList, unixOptions, gnuOptions)
    except getopt.error as err:
        print(str(err))
        sys.exit(2) # Detén la ejecución si los parámetros input son incorrectos

    # Lee los valores de la string con parámetros input
    start_dt = ''
    end_dt = ''
    for currentArgument, currentValue in arguments:
        if currentArgument in ("-s", "--start_dt"):
            start_dt = currentValue
        elif currentArgument in ("-e", "--end_dt"):
            end_dt = currentValue

    # Muestra el resultado en la pantalla.
    print(start_dt, end_dt)

```

Ahora estudiemos el código en detalle. Así es cómo importamos las librerías que necesitamos:

```

import sys
import getopt

```

Después definimos los nombres de los parámetros input:

```

unixOptions = "s:e:"
gnuOptions = ["start_dt=", "end_dt="]

```

- `unixOptions = "s:e:"` define los nombres de los parámetros en estilo clásico **Unix** (Unix es una familia de sistemas operativos desarrollado en los años 70). Aunque la forma en sí es anticuada, se ha convertido en una especie de tradición. El script no se ejecutará sin ello.
- `gnuOptions = ["start_dt=", "end_dt="]` define los nombres de los parámetros de entrada en el estilo de **GNU**, un sistema operativo tipo Unix.

Puedes llamar al script de dos maneras diferentes, dependiendo del estilo de los nombres de los parámetros de entrada:

```

python params_test.py -s '2019-01-01' -e '2019-09-01'
# o
python params_test.py --start_dt='2019-01-01' --end_dt='2019-09-01'

```

Sugerimos que te ciñas a la segunda opción: se lee mejor y garantiza la compatibilidad de parámetros de un programa a otro, para que no mezcles los nombres de las variables.

Entonces el script almacena el conjunto de parámetros de entrada en `argumentList`.

En nuestra muestra los parámetros de entrada son `start_dt` y `end_dt`. El sistema automáticamente los lee en `sys.argv`, que almacenamos en la variable `fullCmdArguments`. Cuando almacenamos en `argumentList`, tomamos todos los parámetros excepto el primero (índice 0). Este es el nombre del script que no necesitamos:

```
fullCmdArguments = sys.argv
argumentList = fullCmdArguments[1:]
```

Entonces el script comprueba si el conjunto de parámetros de entrada está vacío. Si no hay parámetros, el comando `sys.exit(2)` parará la ejecución del programa. `(2)` significa que el error que cause esta parada se hizo en los parámetros de la línea de comandos. Por ejemplo, si los parámetros de entrada se indicaron incorrectamente o no se indicaron en absoluto, entonces el script comprobará si el conjunto de parámetros de entrada está vacío. Si no hay parámetros, el comando `sys.exit(2)` parará la ejecución del programa. `(2)` significa que el error que cause esta parada se hizo en los parámetros de la línea de comandos. Por ejemplo, si los parámetros de entrada fueron indicados incorrectamente o no fueron indicados en absoluto:

```
try:
    arguments, values = getopt.getopt(argumentList, unixOptions, gnuOptions)
except getopt.error as err:
    print (str(err))
    sys.exit(2)
```

A continuación, el script recorre todos los parámetros de entrada y distribuye sus valores entre sus variables internas:

```
start_dt = ''
end_dt = ''
for currentArgument, currentValue in arguments:
    if currentArgument in ("-s", "--start_dt"):
        start_dt = currentValue
    elif currentArgument in ("-e", "--end_dt"):
        end_dt = currentValue
```

Finalmente, el script imprime los valores `start_dt` y `end_dt`.

Lancemos el script en la línea de comandos y pasemos los parámetros de prueba:

```
python params_test.py --start_dt='2019-01-01' --end_dt='2019-09-01'
```

Resultado:

```
[ '--start_dt=2019-01-01', '--end_dt=2019-09-01']  
( '2019-01-01', '2019-09-01' )
```

Puedes pasar cualquier parámetro de entrada al script, incluyendo números, valores lógicos, fechas y horas, matrices y nombres de archivos. Observa que los parámetros solo pueden ser introducidos como strings. Depende completamente de ti convertirlos al formato correcto.

Lanzar un script desde la línea de comando en AWS

Para hacer que un script funcione en una máquina virtual, necesitas transferirlo desde tu máquina local.

Necesitas el programa `scp` (del inglés "secure copy" - copia seguridad) para copiar tus archivos y utilizarlos en tu máquina virtual. Desde la línea de comandos, se lanza de la siguiente forma (recuerda "Ejercicio independiente 1: Configurar una cuenta AWS"):

```
scp -i <path_to_private_key> <path_to_a_local_file> ubuntu@<public_dns>:
```

Para transmitir el archivo a la máquina virtual, sigue estos pasos:

1. Crea un archivo llamado `test.py` y guárdalo en tu PC. Abre el archivo en Sublime Text (u otro editor de texto) y añade el siguiente texto:

```
#!/usr/bin/python  
  
if __name__ == "__main__":  
    print('Hello world.')
```

2. Copia este archivo en la máquina virtual.

Si estás trabajando en Linux, Ubuntu, macOS o Windows 10, utiliza el comando:

```
scp -i <path_to_private_key> <path_to_a_local_file> ubuntu@<public_dns>:
```

Los dos puntos son cruciales: significa "el directorio de inicio del usuario en la máquina virtual." Con otros sistemas operativos, la única diferencia es la forma en la que se indica la ruta de archivo. Asegúrate que sabes cómo encontrar la ruta de un archivo en tu SO.

3. Conéctate a tu máquina virtual:

```
ssh -i <path_to_private_key>/test_pair.pem ubuntu@<public_dns>
```

4. En la línea de comandos de la máquina virtual, ejecuta los comandos para instalar pip (responde Sí a todas las preguntas):

```
sudo apt update
sudo apt install python3-pip
pip3 --version
```

El sistema imprimirá el siguiente resultado:

```
ubuntu@ip-172-31-33-224:~$ pip3 --version
pip 9.0.1 from /usr/lib/python3/dist-packages (python 3.6)
```

5. Una vez conectado, introduce el comando **dir** (directorio). Se te mostrarán los contenidos de tu directorio de inicio:

```
ubuntu@ip-172-31-33-224:~$ dir
test.py
```

6. Comienza el script de prueba con este comando:

```
python3 test.py
```

Programar scripts

Un script es una programa que no solo acepta datos de entrada sino que también puede ser programado. Averigüemos cómo hacer que un script acate un horario de trabajo.

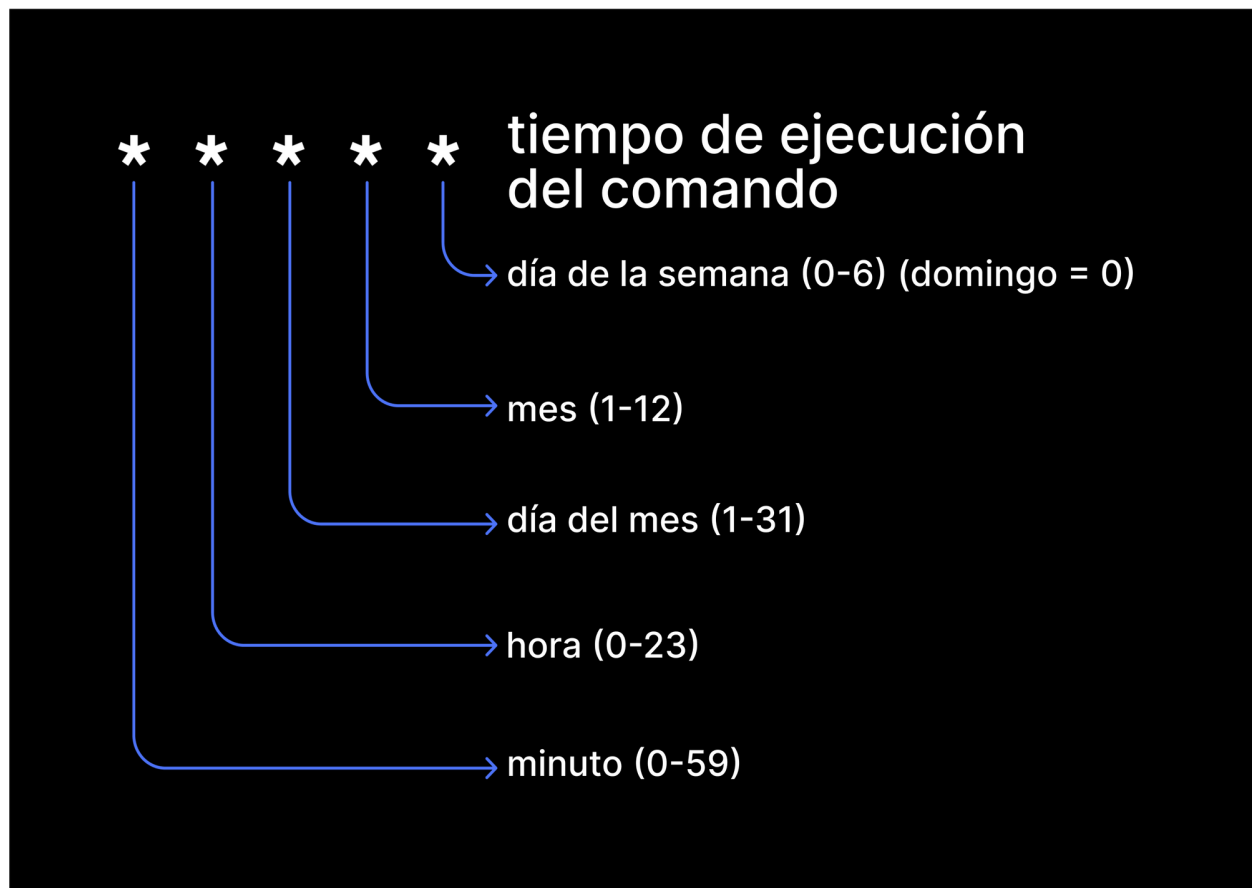
Los sistemas operativos tipo Unix como Linux, Ubuntu y macOS tienen un programa especial de programación llamado **cron**. Funciona de forma invisible en segundo plano y lee una programación especial **crontab** (tab=table).

Aquí está una muestra de la programación **cron** en el terminal:

```
5 6 * * 1 python -u -W ignore /home/my_user/script_A.py --start_dt=$(date +%Y-%m-%d\ 00:00:00 -d "1 week ago") >> /home/my_user/logs/script_A_$(date +%Y-%m-%d).log 2>&1
#15 7 * * * python -u -W ignore /home/my_user/script_B.py --start_dt=$(date +%Y-%m-%d\ 00:00:00 -d "1 week ago") >> /home/my_user/logs/script_B_$(date +%Y-%m-%d).log 2>&1
```

Veamos lo que ocurre. Primero, la primera línea:

- **5 6 * * 1** — **5 6 * * 1** — indica el tiempo que el comando debería ejecutar. Su formato es el siguiente:



* — cualquier valor.

Así que la primera línea especifica que el comando va a empezar cada lunes a las 6:05AM.

- `python -u -W ignore /home/my_user/script_A.py` es el comando en sí. Aquí:
 - El flag `-u` significa que los resultados de la ejecución del script no serán **almacenados** (acumulados en la memoria del PC). En su lugar, se almacenarán inmediatamente en el archivo de registro (más sobre eso más adelante). Por ejemplo, si tu script tiene los comandos `print()`, sus resultados no se almacenarán en la memoria, sino irán directamente a los registros.
 - `-W ignore` (ignorar advertencias) significa que cualquier advertencia generada cuando el script se ejecuta no se almacenará en el archivo de registro.
 - `/home/my_user/script_A.py` — el nombre del script a programar.
- `--start_dt=$(date +%Y-%m-%d\ 00:00:00 -d "1 week ago")` — el parámetro de entrada del script.

Aquí:

- `$(date +%Y-%m-%d\ 00:00:00 ... )` — la expresión de línea de comandos que te permite conseguir la fecha actual en el formato `'%Y-%m-%d 00:00:00'`. Para entender mejor cómo funciona, ejecuta el siguiente comando en la interfaz de líneas de comandos:

```
echo $(date +%Y-%m-%d\ 00:00:00)
```

Verás la fecha actual en la pantalla.

- `-d "1 week ago"` especifica el intervalo de tiempo por el que queremos disminuir la fecha actual. `script_A.py` parece estar diseñado para que su entrada sea la fecha en la que comenzó la semana anterior. Probablemente recopila ciertos datos de los siete días anteriores. Hay también otras formas de definir los intervalos de tiempo. Por ejemplo:
 - `-d "yesterday"` o `-d "1 day ago"`
 - `-d "N days ago"`
 - `-d "N weeks ago"`
 - `-d "1 month ago"`
 - `-d "N months ago"`
 - `-d "1 year ago"`
 - `-d "N years ago"`
- `>> /home/my_user/logs/script_A_$(date +%Y-%m-%d).log` significa que todos los datos que imprime el script se almacenarán en el archivo `script_A_complianceDate.log` en `/home/my_user/logs/`. Es muy importante guardar los registros si los scripts se ejecutan automáticamente; esto te ayudará a detectar cualquier error que se produzca. Sin registros no tendrás una idea clara de cómo el sistema de automatización funciona realmente.
- `2>&1` significa que todos los resultados de la ejecución del script (incluidos los errores) serán impresos en el mismo lugar: el archivo de registro.

El tiempo actual se determina por la zona horaria de la máquina donde vas a adaptar cron. La mayoría de los servidores trabajan con la zona horaria `UTC+0`. Esta zona horaria se utiliza generalmente cuando grabamos la fecha y hora de las operaciones bancarias y transacciones. Trata de ceñirte a `UTC+0` cuando analices y programes scripts para evitar discrepancias en los resultados del informe.

Veamos cómo editar el horario `cron`. Solo puedes hacerlo en una máquina local con Linux, Ubuntu o macOS.

1. Primero, crea un archivo vacío en Sublime Text que imprimirá `'Hello world'` y la fecha y hora actuales:

```
#!/usr/bin/python

from datetime import datetime

if __name__ == "__main__":
    print('Hola, mundo: {}'.format(datetime.now()))
```

2. Lo guardaremos como `/home/YOUR_USERNAME/cron_test.py`.

3. Después ejecutaremos el siguiente comando en la línea de comandos:

```
python cron_test.py
```

Resultado:

```
15:14 ~ $ python cron_test.py
Hello world: 2019-10-01 15:15:10.975797
```

En la línea de comandos crearemos el directorio `/home/YOUR_USERNAME/logs` para almacenar registros:

```
mkdir /home/YOUR_USERNAME/logs
```

A continuación, llamaremos al editor de programación `cron`:

```
crontab -e
```

Tendrás que elegir un editor de texto la primera vez que edites tu programación:

```
yandex_cloud@automation-test:~$ crontab -e
no crontab for yandex_cloud - using an empty one

Select an editor. To change later, run 'select-editor'.
 1. /bin/nano          <---- easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny
 4. /bin/ed
```

Teclea "1" y pulsa Intro para elegir nano como el editor de texto para crontab.

Después verás el texto crontab:

```
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
```

Get Help Write Out Where Is Cut Text Justify Cur Pos Prev Page First Line
Exit Read File Replace Uncut Text To Spell Go To Line Next Page Last Line

Añade la siguiente línea al final del archivo:

```
*/5 * * * * python -u -W ignore /home/YOUR_USERNAME/cron_test.py >> /home/YOUR_USERNAME/logs/cron_test.log 2>&1
```

Aquí `/5 * * * *` significa "ejecutar cada cinco minutos."

```
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
*/5 * * * * python -u -W ignore /home/YOUR_USER/cron_test.py >> /home/YOUR_USER/logs/cron_test.log 2>&1
```

Presiona `Ctrl+O` para guardar. Verás el mensaje del sistema:

```
File Name to Write: /tmp/crontab.inCPUi/crontab
^G Get Help      M-D DOS Format    M-A Append       M-B Backup File
^C Cancel        M-M Mac Format    M-P Prepend      ^T To Files
```

Presiona Intro. Después presiona `Ctrl+X` para quitar el editor crontab. Verás el mensaje:

```
15:25 ~ $ crontab -e  
crontab: installing new crontab  
15:48 ~ $
```

Esto significa que los nuevos ajustes para la tabla cron se han adoptado.